Session 7 : Block, File & Object Storage

RECAP: Session 6 : Load Balancing & CDN

1. Load Balancing Basics: Purpose: availability & scalability; the single-point-of-failure problem
2. Load Balancer Types: Layer 4 (transport) vs Layer 7 (application); internal vs internet-facing
3. Distribution Algorithms: Round robin; least connections; IP hash; weighted distribution
4. Health Checks & Sessions: Health check design; draining an unhealthy backend; sticky sessions vs stateless design
5. DNS & Content Delivery: DNS-based routing & failover; CDN: edge caching, origin & cache invalidation

Agenda for Today's Session

1. Block Storage: Volumes attached to instances; IOPS & throughput; use cases
2. File Storage: Shared filesystems: NFS & SMB; when file storage beats block
3. Object Storage: Buckets, objects, keys & metadata; the flat namespace
4. Object Storage Mechanics: REST API access; presigned URLs; versioning
5. Choosing a Storage Type: Comparison across performance, sharing, scale & cost

---------- SECTION 1: 1. Block Storage: Volumes attached to instances; IOPS & throughput; use cases ----------

Slide 1: Block Storage Explained

Heading: Block Storage Explained

Subheading: Raw Disks For Instances

Content:

A block storage volume is a raw disk that an operating system partitions and formats itself. Amazon Elastic Block Store (EBS) is AWS's block storage service, giving each EC2 instance a persistent virtual disk.

- An EBS volume attaches to only one EC2 instance at a time.
- A volume lives in one Availability Zone and attaches only there.
- Volume data persists independently of the instance, surviving stop, start or termination.
- EBS volumes come in SSD-backed and HDD-backed families for different workloads.

Analogy: An external hard disk plugged into one laptop stores files only that laptop can read directly — just as an EBS volume attaches as a raw block device to a single EC2 instance at a time.

Visual Guidance: Diagram: EC2 instance icon inside an Availability Zone box, with an attached disk icon labelled EBS Volume beside it.

Speaker Notes: "Explain that EBS gives an instance its own virtual disk, separate from its lifecycle. Note that the Availability Zone lock is a physical constraint, not a setting. Mention this will later contrast with shared file storage; interviewers often ask why EBS cannot attach across zones — the answer is physical replication boundaries."

Slide 2: IOPS and Throughput Defined

Heading: IOPS And Throughput

Subheading: Measuring Volume Performance

Content:

Input/Output Operations Per Second (IOPS) counts individual read/write requests a volume completes per second, while throughput measures total data moved per second in megabytes. Together they describe volume speed from two angles.

- IOPS matters most for small, random reads and writes, such as transaction logs.
- Throughput matters most for large, sequential transfers, such as video streaming or backups.
- General purpose SSD volumes offer baseline IOPS and throughput that scale somewhat with volume size.
- Provisioned IOPS volumes let an application request guaranteed IOPS, independent of volume size.

Visual Guidance: Two-column visual: left shows many small IOPS arrows into a disk, right shows one wide throughput arrow.

Speaker Notes: "Explain IOPS and throughput as two separate speedometers. Contrast a database's many small writes with a video backup's one large sequential write, and note that optimising for the wrong metric is a common mistake. Interviews often ask which metric matters most for a given workload."

Slide 3: EBS Volume Types Compared

Heading: EBS Volume Types

Subheading: SSD Versus HDD Backing

Content:

Different workloads need different performance profiles, so Amazon EBS offers volume types split into two backing media: solid-state drives for speed and hard-disk drives for lower-cost bulk transfer.

Volume Type	Backing Media	Best Fit
General Purpose SSD (gp3)	SSD	Boot volumes and most general workloads
Provisioned IOPS SSD (io2)	SSD	Databases needing guaranteed, consistent low latency
Throughput Optimized HDD (st1)	HDD	Big sequential workloads like log processing
Cold HDD (sc1)	HDD	Infrequently accessed data at lowest cost

A database requiring 15,000 IOPS and 2,000 GiB of storage can use either gp3 or io2 without increasing storage capacity to achieve the required IOPS. gp3 offers a cost-effective option, while io2 suits mission-critical workloads requiring higher durability and consistently low latency.

Visual Guidance: Render the four-row comparison table with the two SSD types grouped above the two HDD types.

Speaker Notes: "Walk through the table, noting the SSD-versus-HDD split first. Mention gp3 is the default choice for most workloads, while io2 trades cost for guaranteed performance. Interviewers may ask which volume type suits a workload — match the access pattern to SSD or HDD first."

Slide 4: Where Block Storage Is Used

Heading: Block Storage Use Cases

Subheading: Where EBS Fits Best

Content:

Choosing a volume type means matching an application's access pattern to the right backing media, since the same workload on the wrong type either wastes money or starves for performance.

- Boot volumes store the OS and startup files, typically on general purpose SSD.
- Relational databases need consistent low-latency IOPS, favouring provisioned IOPS SSD.
- Big data and log-processing pipelines read large sequential files, favouring throughput-optimized HDD.
- Archival or rarely-touched data favours cold HDD for lowest cost.

A nightly 500 GB log pipeline fits st1, priced per gigabyte with no per-IOPS charge. The same workload on io2's 15,000 IOPS would cost far more for unneeded throughput.

Visual Guidance: Four icons — boot disk, database, log files, archive box — each pointing to its matching volume type from the previous slide.

Speaker Notes: "Map each use case to the volume type from the previous table. Contrast a database transaction with a nightly log dump, noting that EBS pricing scales with provisioned IOPS and throughput. Interview questions often ask which volume type fits a given workload."

Slide 5: Block Storage Trade-offs

Heading: Trade-offs And Limits

Subheading: Cost And AZ Limits

Content:k

Block storage's performance comes with real constraints: a volume is locked to one Availability Zone and normally serves only one instance, so cost and location choices must be made upfront.

- Provisioned IOPS volumes cost more, so size workloads to actual need.
- A volume cannot attach across Availability Zones without a snapshot copy.
- Normally only one instance can mount a volume at a time.
- Over-provisioned IOPS or throughput sits idle, so right-sizing avoids paying for unused headroom.

Visual Guidance: Short icon row: a locked Availability Zone padlock, a single-instance plug icon, and a cost-tag icon.

Speaker Notes: Stress that the Availability Zone lock is physical, not configurable; plan volume placement accordingly. Higher guaranteed IOPS cost more per gigabyte. Flag the single-attach limit as why shared file storage exists, without naming it yet; interviewers ask why an EBS volume cannot move between zones.

---------- SECTION 2: 2. File Storage: Shared filesystems: NFS & SMB; when file storage beats block ----------

Slide 6: File Storage Overview

Heading: File Storage Basics

Subheading: Shared Filesystem Landscape

Content:

File storage provides a shared filesystem that multiple servers mount like one common drive, instead of separate private disks. NFS and SMB carry this access.

- NFS originated on Unix and Linux, and remains the Linux standard.
- SMB (Server Message Block): originated on Windows, is the Windows file-share standard.
- Both protocols allow multiple clients to read and write the same files concurrently.

Visual Guidance: Diagram: several server icons connected to one shared filesystem box, labelled NFS (Linux) and SMB (Windows) as the two access protocols.

Speaker Notes: "Introduce file storage as the shared-drive model before naming NFS and SMB individually. Mention that AWS offers this as Amazon EFS and Amazon FSx, covered next. Exam hook: learners should recall that file storage means several machines mount the same filesystem at once."

Slide 7: NFS: Network File System

Heading: NFS Mechanism

Subheading: How NFS Works

Content:

Network File System (NFS) lets a client mount a remote directory so it behaves as local, while data stays on the server. The client sends read, write and lock requests for the server to execute.

- NFSv3 is stateless, so each request carries full file-handle and offset information.
- NFSv4 is stateful, adding built-in locking, delegation, and improved security over TCP.
- Exports define which server directories are shared and their permissions.
- Concurrent writers to the same file need explicit locking to avoid corruption.

Analogy: An office notice board lets every employee read and pin updates on the very same board instead of each keeping a private copy — just as NFS lets every mounting client read and write the one shared copy of a file kept on the server.

Visual Guidance: Diagram: client box with a mounted folder icon pointing via an arrow labelled 'NFS protocol' to a server box holding the real files.

Speaker Notes: "Explain that mounting makes a remote folder look local to the application. Point out the stateless-versus-stateful split between NFSv3 and NFSv4 as a common interview question. Mention that Amazon EFS is built on the NFSv4.1 protocol."

Slide 8: SMB: Server Message Block

Heading: SMB Mechanism

Subheading: Windows File Sharing

Content:

Server Message Block (SMB) is the Windows protocol for sharing files, printers and named pipes over a network, exposing each shared folder as a network share.

- SMB authenticates each session against Windows user or domain credentials.
- Opportunistic locks: a client caches a file locally and gets notified of changes.
- SMB 3.0 added end-to-end encryption and multichannel transport for higher throughput.
- CIFS: the older, less efficient dialect that SMB replaced.

Visual Guidance: Diagram: Windows client icon connecting over SMB to a network share folder on a file server, with an authentication step shown before access is granted.

Speaker Notes: "Contrast SMB's session-based authentication with NFS's export-based permissions covered earlier. Note that Amazon FSx for Windows File Server offers SMB-based storage in AWS. Exam hook: ask which protocol pairs with which operating system family."

Slide 10: When File Storage Wins

Heading: File Vs Block

Subheading: Choosing File Storage

Content:

Block storage attaches to one instance at a time, so an application that needs several servers to read and write the same files needs a shared filesystem instead.

- Content management systems and shared home directories need concurrent multi-client access.
- Rendering or batch pipelines let many worker nodes share intermediate files.
- Amazon EFS scales automatically but costs more per gigabyte than object storage.

Visual Guidance: Decision diagram: single-instance workload arrow pointing to block storage; multi-instance shared-access workload arrow pointing to file storage (EFS/FSx).

Speaker Notes: "Emphasise that the deciding question is whether more than one server must see the same files at once. Mention Amazon EFS's per-gigabyte pricing as the cost trade-off for that convenience. Exam hook: ask learners to justify choosing EFS over EBS for a specific shared workload."

---------- SECTION 3: 3. Object Storage: Buckets, objects, keys & metadata; the flat namespace ----------

Slide 11: Object Storage Landscape

Heading: Object Storage Map

Subheading: Buckets To Namespace

Content:

Object storage keeps data as independent units, not files in folders or disk blocks, used for backups and media at scale. Amazon S3 is the AWS service used here.

- Bucket: top-level container holding objects, setting region, access and lifecycle rules.
- Object: one stored item, made of the actual data, a key and metadata.
- Key: the unique string that identifies an object inside its bucket.
- Metadata: descriptive information attached to an object, kept separate from the data itself.
- Flat namespace: all objects sit at one level, no folder tree.

Visual Guidance: Diagram: a bucket icon containing several small object icons, each tagged with a key string and a metadata label; beside it, a flat list of keys contrasted with a traditional folder tree.

Speaker Notes: "Open with the landscape before covering bucket, object, key, metadata and flat namespace. Name Amazon S3 as the AWS service so terms attach to a real product. Flag flat namespace as the structural point later slides revisit."

Slide 12: What An Object Is

Heading: Object Anatomy

Subheading: Data Key Metadata

Content:

An object is the basic unit of object storage: data, a key and metadata bundled together. Unlike a disk file, an object keeps all three parts together wherever it is stored.

- Data: the actual bytes of the object, stored as an opaque blob.
- Key: a string that uniquely names the object inside its bucket.
- System metadata: content type, size and last-modified date, maintained automatically.
- User metadata: custom key-value pairs that an application attaches.

Visual Guidance: Diagram: one object box split into three labelled parts — Data, Key, Metadata — with example values filled in for a sample file.

Speaker Notes: "Explain that every object bundles data, key and metadata together, unlike a plain disk file. Use the courier parcel analogy: package, tracking label, contents declaration. Note that system metadata is automatic while user metadata is custom, and ask which one a user can actually edit."

Slide 13: How Buckets Work

Heading: Bucket Mechanics

Subheading: Naming And Scope

Content:

A bucket lives in one AWS region, and its name must be unique across all of AWS, since S3 uses it in every object's address. Objects inherit that region by default.

- Global uniqueness: no two AWS buckets can share the same name.
- Naming rules: lowercase letters, numbers and hyphens, three to sixty-three characters long.
- Region pinning: a bucket stays in its chosen AWS region.
- Object count: a bucket can hold unlimited objects, with no folder limit.

Visual Guidance: Bucket icon labelled with its globally unique name and region tag, with a crossed-out duplicate name shown in a different account to illustrate global uniqueness.

Speaker Notes: "Stress that bucket names compete globally, unlike folder names that need uniqueness only within a parent. Mention that region choice affects latency and data location. A common exam trap is assuming bucket names are scoped to one account."

Slide 14: Flat Namespace Vs Folders

Heading: No Real Folders

Subheading: Prefixes Simulate Trees

Content:

A file system nests folders inside folders, but an object store keeps every object flat, addressed only by its key. Console tools fake folder views by splitting keys on a slash.

Aspect	Hierarchical File System	Flat Object Namespace

Structure	Nested folders and sub-folders	Single flat list of keys per bucket
Path meaning	Directory entries physically group files	Key string only; the slash is just a character
Listing a folder	Operating system walks the directory tree	Client filters keys sharing one prefix
Renaming a folder	Updates a single directory entry	Means rewriting every key sharing that prefix

Visual Guidance: Side-by-side diagram: a nested folder tree on the left, and on the right a flat bucket with keys like photos/2024/a.jpg shown as plain strings, with a dashed line grouping keys that share a prefix.

Speaker Notes: "Explain that S3 console folders are an illusion built from key prefixes, not real directories. Renaming a folder path is costly: every object sharing the prefix must be copied and rewritten. This design scales S3 to billions of objects; interviewers ask why renaming isn't instant."

---------- SECTION 4: 4. Object Storage Mechanics: REST API access; presigned URLs; versioning ----------

Slide 16: How Applications Access Object Storage

Heading: Object Storage Mechanics

Subheading: Three Access Mechanisms

Content:

Object storage exposes buckets and objects through a web interface, not a mounted drive. Three mechanisms govern access and control who can reach the data.

- REST API access: every read, write or delete runs as an HTTP request.
- Presigned URLs: a signed, time-limited link grants access without sharing permanent AWS credentials.
- Versioning: a bucket can keep every prior copy of an object under one key.

Visual Guidance: Diagram: a client box sending an HTTP request to an S3 endpoint, branching into three labelled paths — REST call, presigned link, versioned object stack.

Speaker Notes: "Frame this as the mechanics behind the bucket-and-key model already taught: how something actually reaches an object. Mention that each of the three gets its own slide next. Interviewers often ask how S3 is accessed without a filesystem mount — this slide is the answer."

Slide 17: REST API Access To S3 Objects

Heading: REST API Access

Subheading: HTTP Verbs And Auth

Content:

Representational State Transfer (REST) access means every S3 operation is a plain HTTP request to the object's own URL. The console, CLI and SDKs are just wrappers around these same calls.

- GET retrieves an object; PUT uploads or overwrites it; DELETE removes it.
- Requests to private data are signed with AWS Signature Version 4, not passwords or cookies.
- The request URL includes the bucket name and object key as the resource path.
- An unsigned or wrongly signed request to private data is rejected with an HTTP 403 error.

Analogy: At a bank, you give the locker number and say what you want to do (open, put in, or close it), and the bank checks your ID before acting, just as a REST call names the object key and HTTP verb, and S3 checks the signature before acting.

Visual Guidance: Show a sample request line: GET /bucket-name/object-key with an Authorization header, pointing arrows from 'verb' to action and 'path' to the object.

Speaker Notes: "Say there is no drive letter, only a web address and a verb, and this is how every SDK call translates. Exam hook: ask what distinguishes GET from PUT on the same URL — the verb, not the address."

Slide 18: Presigned URLs For Temporary Access

Heading: Presigned URLs

Subheading: Temporary Signed Access

Content:

A presigned URL is a signed link giving temporary access to a private object without sharing AWS credentials. Anyone with access to the object can generate it, and it works until it expires.

- The signature is computed from owner credentials, object key, and expiry timestamp.
- S3 checks the signature and expiry on each request and rejects expired links, with no app server involved.
- A browser or mobile app can upload or download directly, skipping the server.
- Validity windows typically run from minutes to several days, set at generation.

Analogy: A movie ticket booked online gives you a QR code that lets you into one show at one time, without making you a theatre staff member. Once the show time passes, the QR code stops working, just as a presigned URL lets its holder reach one object for a limited time without ever receiving the owner's AWS credentials.

Visual Guidance: Timeline graphic: link generated at T0, valid window shaded, expiry marker at T+n, with an 'access denied' icon after expiry.

Speaker Notes: "Explain this as sharing a private file without making the bucket public. Mention the common use case of an app letting users upload directly to S3. Exam hook: ask who enforces the expiry — AWS, not the application — and why that matters for security."

Slide 19: Versioning Keeps Every Object Copy

Heading: Object Versioning

Subheading: Keeping Prior Copies

Content:

Versioning lets a bucket keep every prior copy of an object under the same key instead of overwriting it. Once enabled, each PUT creates a new version with its own version ID.

- A GET without a version ID returns the current version.
- Deleting an object adds a delete marker, not erasing earlier versions.
- Any version ID can be restored via explicit REST API request.
- Retaining every version raises storage cost, since each overwrite is billed separately.

Visual Guidance: Stack diagram: one object key with three stacked version blocks (v1, v2, v3-current), a delete-marker block on top of v3, and an arrow restoring v2 to the top.

Speaker Notes: "Tie this to the earlier point that a key is a flat name — versioning adds hidden depth behind it. Mention accidental-deletion recovery as the main reason teams enable it. Exam hook: ask what happens to storage cost if versioning stays on indefinitely."

---------- SECTION 5: 5. Choosing a Storage Type: Comparison across performance, sharing, scale & cost ----------

Slide 20: Four Dimensions Of Storage Choice

Heading: Four Decision Dimensions

Subheading: Performance Sharing Scale Cost

Content:

Block, file and object storage were each built around a different design goal, so picking one for a workload comes down to four questions rather than a single rule.

- Performance: how fast single operations complete, measured as IOPS, throughput or latency.
- Sharing: whether one instance or many clients can read and write the same data at once.
- Scale: how large the data can grow and how that growth is managed.
- Cost: whether billing follows provisioned capacity or actual consumption.

Visual Guidance: Four-quadrant diagram with Performance, Sharing, Scale, Cost as labels, each pointing to the storage family it weighs heaviest for.

Speaker Notes: "Frame this as the exam-style question: given a workload, which storage type fits. State each dimension in plain words before the comparison table. Interviewers often ask how to choose between S3, EBS and EFS — this slide is the answer skeleton."

Slide 21: Storage Types Across The Four Dimensions

Heading: Storage Types Compared

Subheading: Across Four Dimensions

Content:

S3, EBS and EFS differ across performance, sharing, scale and cost, along with how they are accessed and how long data lasts, so each suits a different kind of workload.

Feature	Amazon S3	Amazon EBS	Amazon EFS
Storage type	Object storage	Block storage	File storage
Access method	Console, CLI or REST API	Block device formatted and mounted by the OS	Network file system mounted by clients
Performance	Higher latency, very high throughput	Lowest latency, high IOPS	Low-millisecond latency, throughput scales with use
Shared access	Many clients via permissions, no shared filesystem	Normally one instance at a time, same Availability Zone	Many EC2 instances mount concurrently
Scale	Virtually unlimited, automatic	Fixed size, can be increased manually, not shrunk	Grows and shrinks automatically

Cost	Pay per GB used plus requests, lowest per GB	Pay for provisioned size, even if unused	Pay per GB used, higher per GB
Persistence	Until deleted or removed by a lifecycle rule	Independent of the instance; root volumes deleted on termination by default	Independent of client instances until deleted
Best suited for	Images, documents, backups, downloadable files	Boot disks and additional instance storage	Shared directories across multiple servers

Visual Guidance: Render the comparison table with the four agenda rows (Performance, Shared access, Scale, Cost) lightly highlighted.

Speaker Notes: Walk the four highlighted rows first: performance, sharing, scale and cost, since these decide the choice. Then use the other rows for context. Interview hook: given a workload, ask learners to pick S3, EBS or EFS and justify it using these rows. A common trap is calling one type simply ‘better’ instead of the right fit for the workload.”

Slide 22: Matching Workload To Storage Type

Heading: Matching Workload Needs

Subheading: Selection Criteria

Content:

Translating the table into a decision means matching the row that matters most for the workload to the storage type that wins on it.

- Amazon S3 suits objects accessed by key via console, CLI or API.
- Amazon EBS suits one instance needing disk-like storage in its Availability Zone.
- Amazon EFS suits multiple instances that read and write shared files.
- Selecting a service follows the application's access pattern, not habit or familiarity.

Visual Guidance: Decision flow: workload question at top branching into three boxes (block, file, object), each with one real example label underneath.

Speaker Notes: "Map each service to one example: EBS for a database's data files, EFS for a shared config directory, S3 for user-uploaded images. S3 stores objects, EBS gives block volumes, EFS gives a shared file system. Expect interview questions like 'design the storage layer for X'."

---------- PRACTICAL HANDS-ON: Testing S3, EBS and EFS Storage on AWS ----------

Slide 24: Lab Overview And Major Steps

Heading: Lab Overview

Subheading: What Gets Tested

Content:

This practical builds one example of each storage type: an S3 bucket for objects, an EBS volume on one instance for block storage, and an EFS file system shared between two instances for file storage.

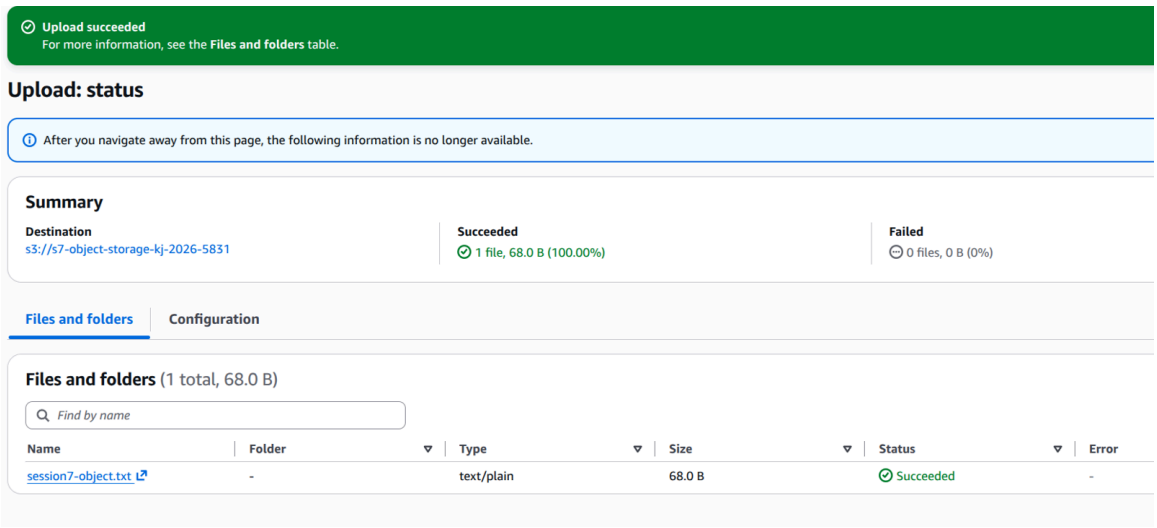
- Store an object in S3 and confirm it downloads with the same content.
- Attach an EBS volume to one instance, format and mount it, then store a file on it.
- Create a shared EFS file system reachable from two separate instances.
- Write a file on one instance, read it from the other.
- Compare all three services on performance, sharing, scale and cost.

Slide 25: Object Storage Evidence in S3

Heading: S3 Upload Verified

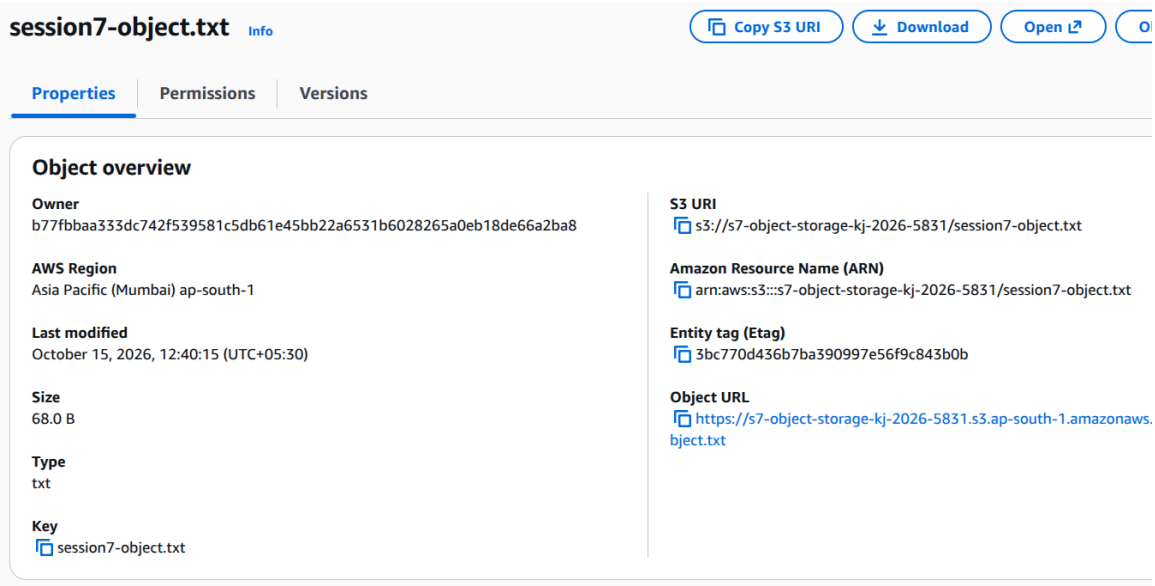
Subheading: Properties Confirm Object

Content:



S3 upload completion summary

The upload summary confirms one file succeeded, with zero failures, uploaded to the bucket destination shown in the S3 URI. This is the moment the object becomes available by its key.



S3 object properties for session7-object.txt

The Properties tab lists the object key, its size, type, region and the S3 URI used to address it, confirming the object and its metadata are stored and retrievable.

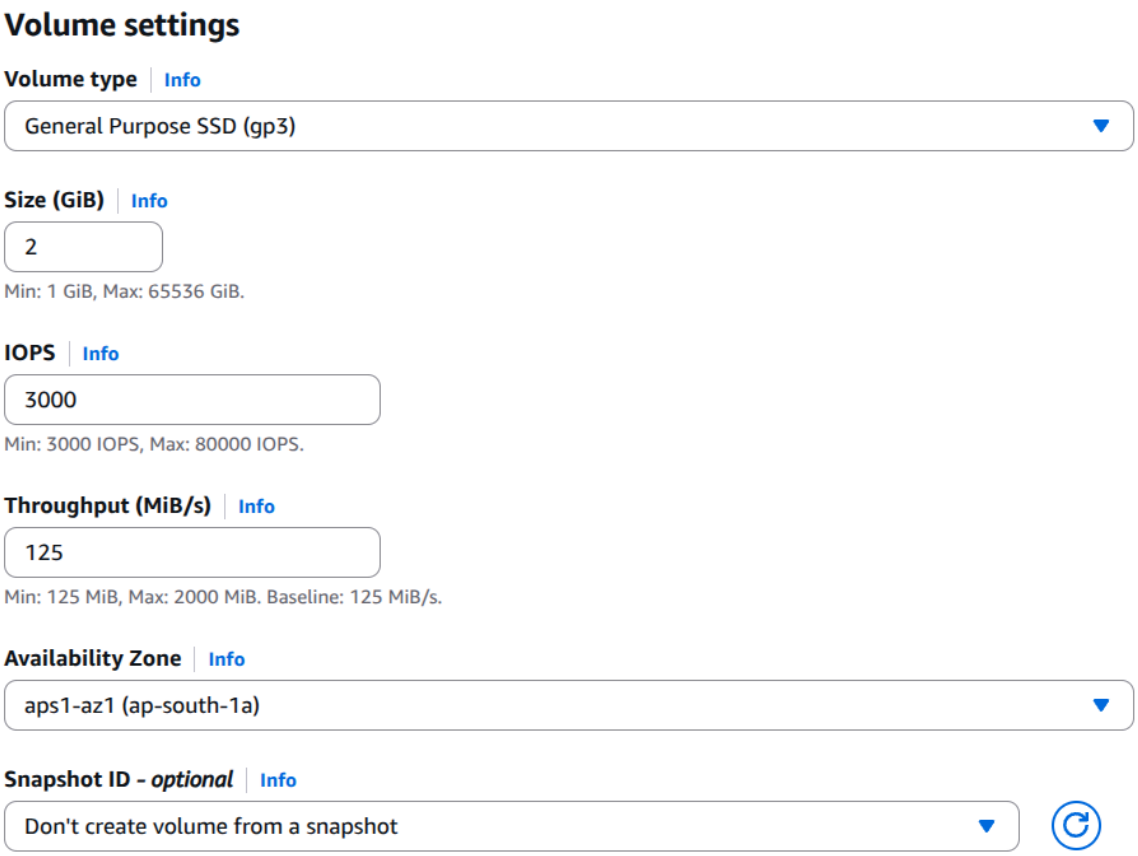
Slide 26: Block Storage Setup and Attachment

Heading: EBS Configuration

Subheading: Volume Matches AZ

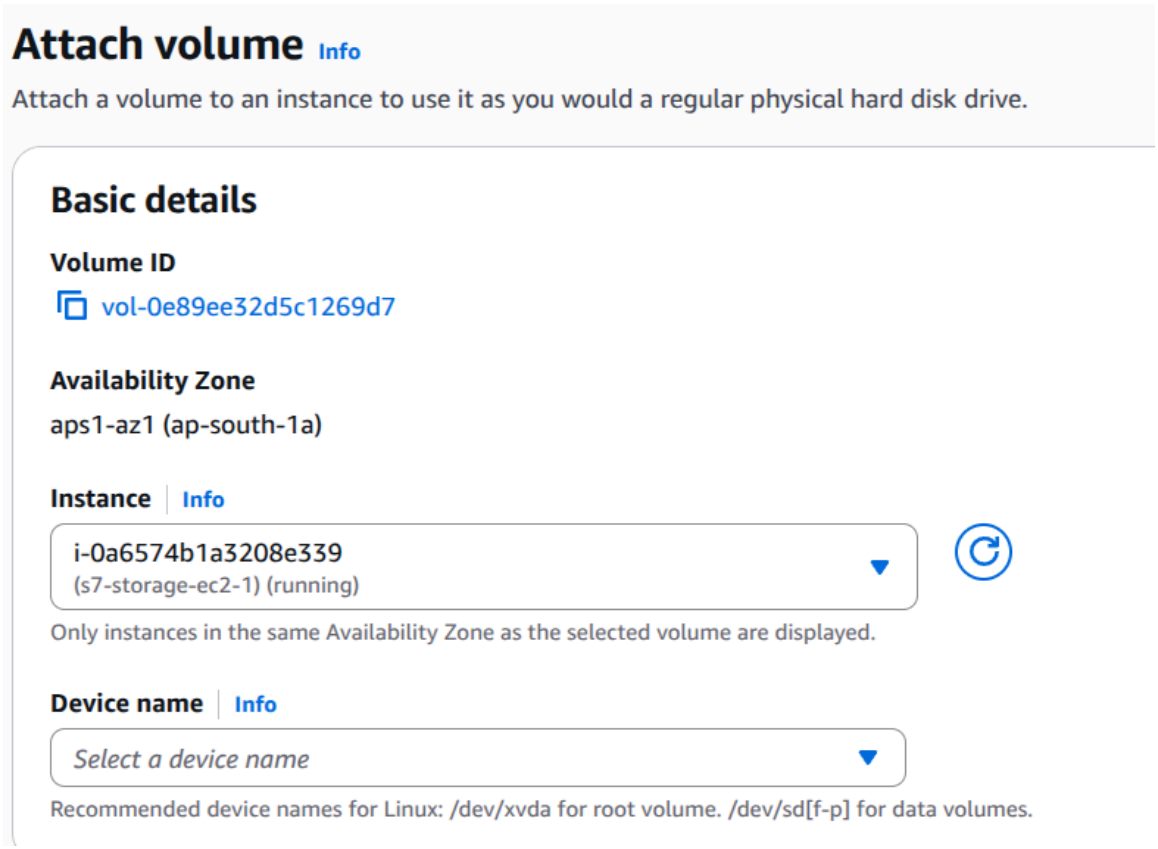
Content:

An EBS volume and the instance it attaches to must sit in the same Availability Zone, since a volume cannot attach across zones; this pairing is checked before creation, not after.



EBS volume settings before creation

The volume settings show a gp3 volume sized 2 GiB created in the same Availability Zone as the instance, matching the rule that a volume and its instance must share a zone.



Attach volume dialog with instance selected

The attach dialog pairs the volume ID with the running instance, both shown in the same Availability Zone, which is the precondition for a successful attachment.

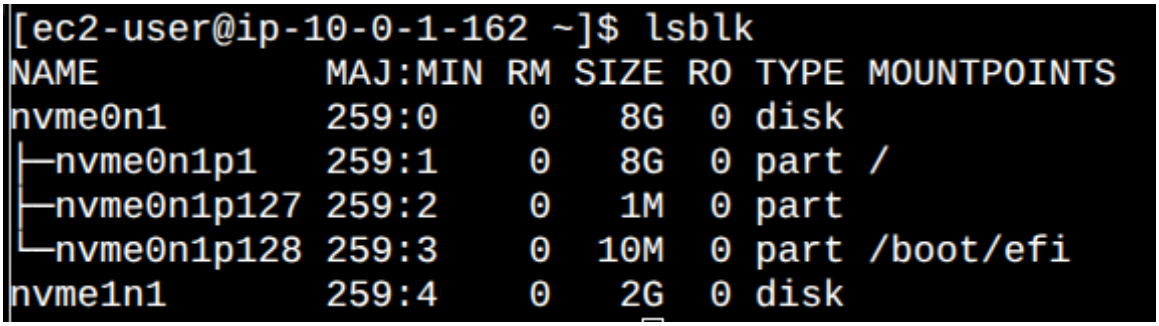
Slide 27: Identifying and Verifying the EBS Disk

Heading: Disk Identified

Subheading: Format And Persist

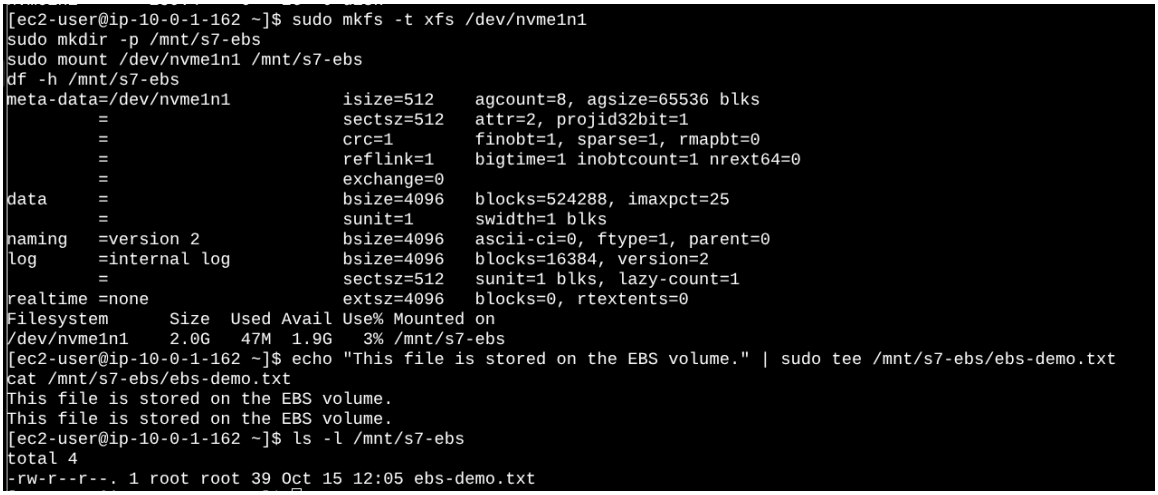
Content:

Formatting a disk erases anything already on it, so the newly attached empty disk must be identified by its size before any format command runs, and the root disk is never touched.



lsblk output listing root and data disks

The lsblk output lists the root disk nvme0n1 with its boot partitions alongside the separate 2 GiB disk nvme1n1, which has no partitions yet and is the empty volume to format; device names can differ from what the console showed.



EBS volume formatted, mounted, file written

Disk formatted, mounted, and a file written and read back from the EBS volume.

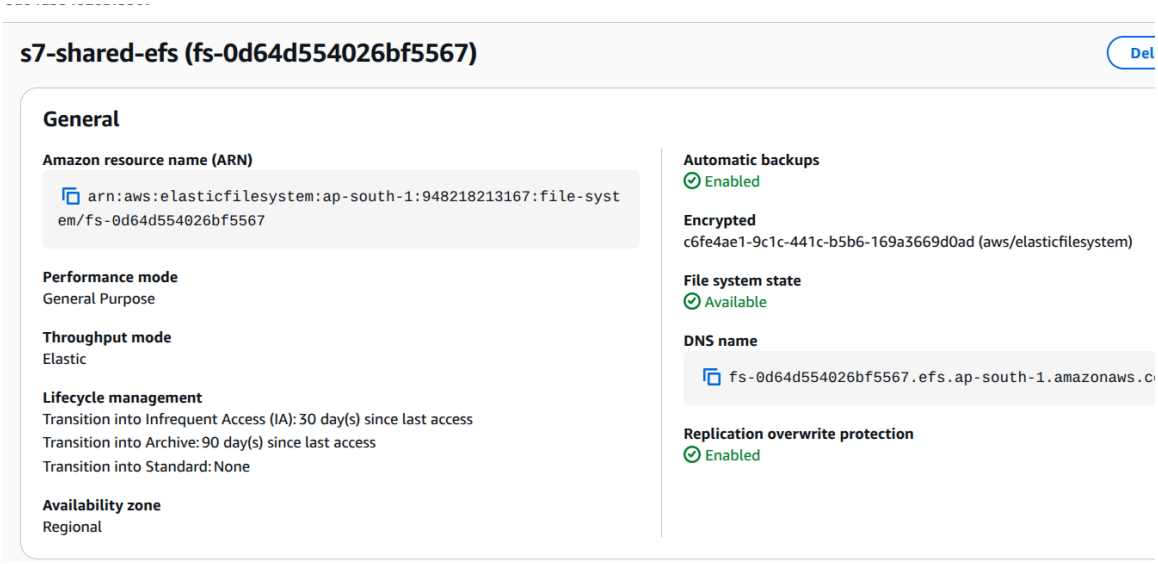
Slide 28: Building the Shared EFS File System

Heading: EFS Creation

Subheading: Mount Target Ready

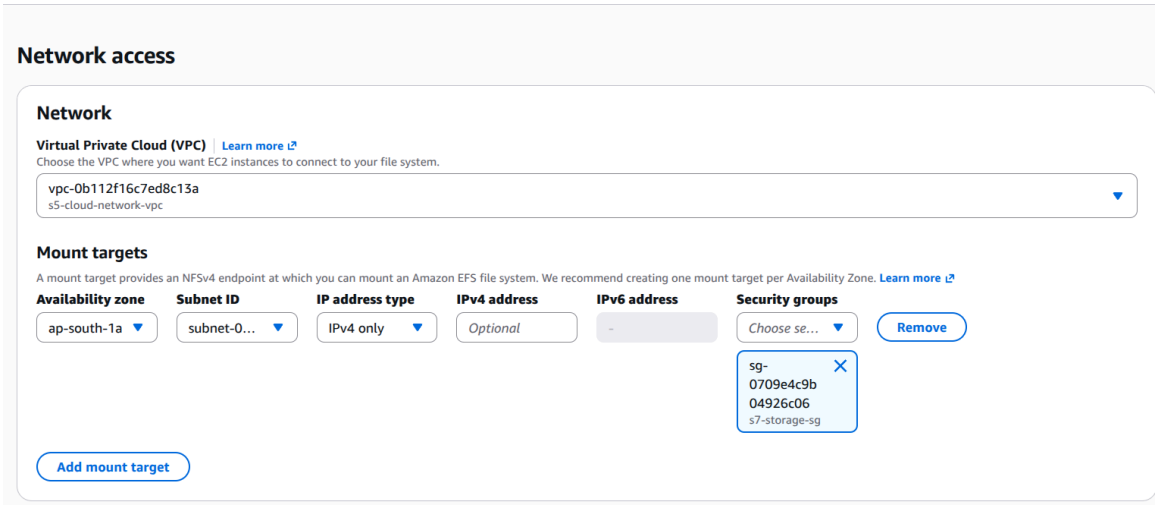
Content:

Before mounting, the file system ID, the DNS name, the mount target's state and the security group rule allowing network file system traffic are all verified on both instances.



EFS file system general settings

The file system details confirm it was created with General Purpose performance and Elastic throughput and shows an available state; this file system ID is what both instances must mount using the identical value.



EFS mount target network configuration

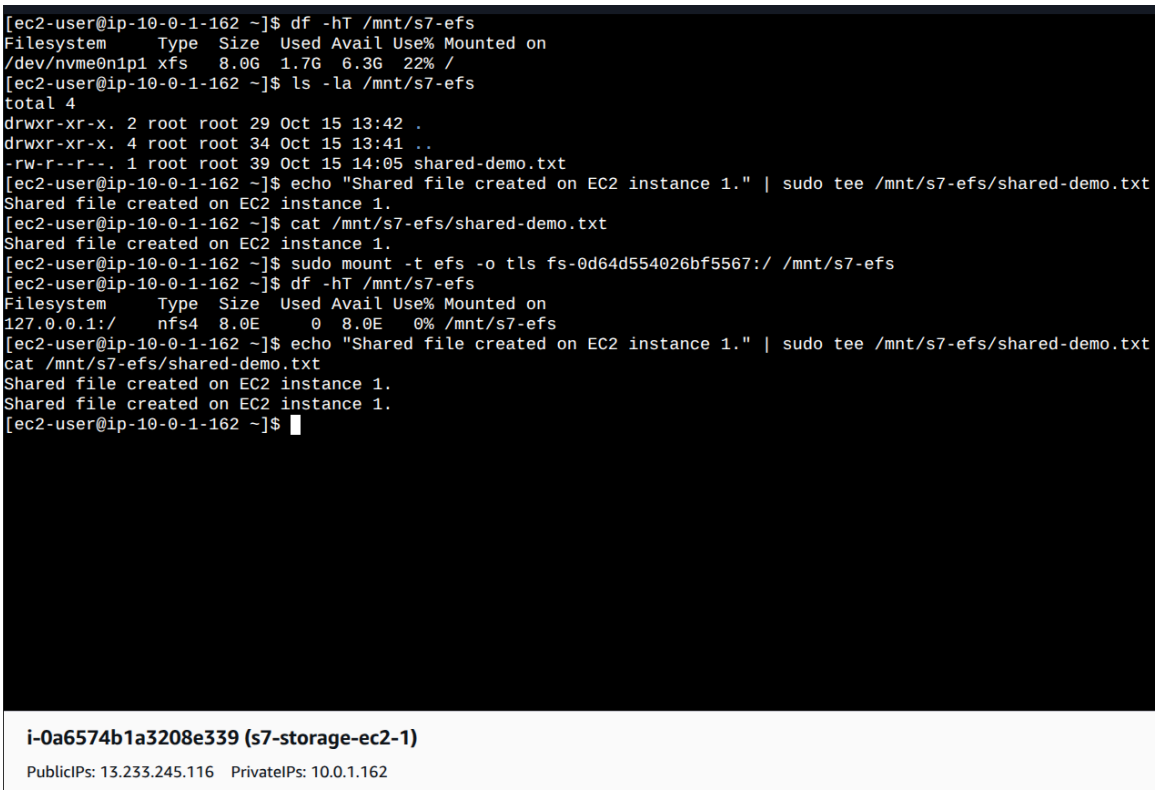
The network access section shows the mount target placed in the same VPC and Availability Zone as the instances, with a security group permitting NFS traffic from the instances.

Slide 29: Sharing, Diagnostics And Cleanup

Heading: Sharing And Cleanup

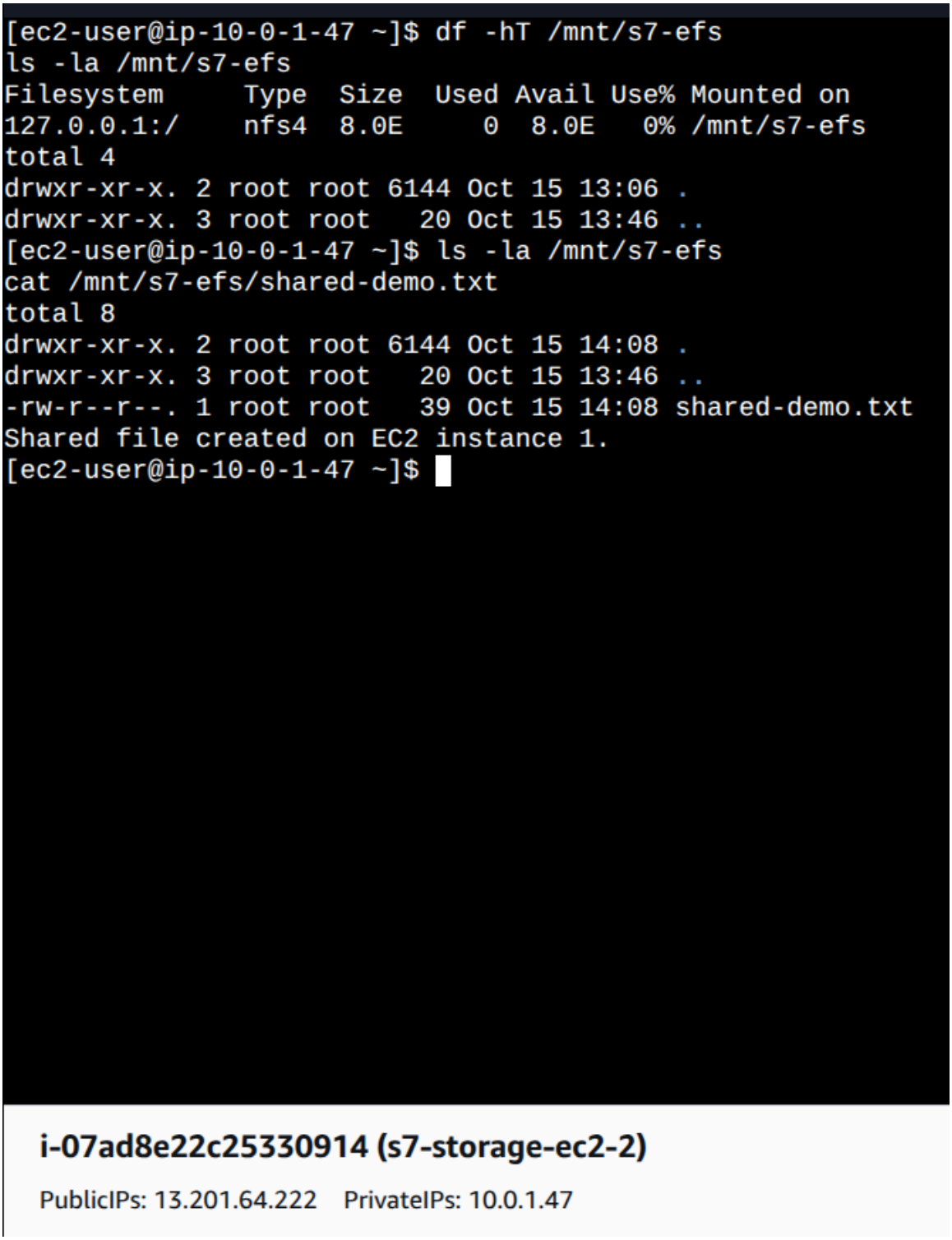
Subheading: Verify Then Remove

Content:



Instance 1 terminal showing EFS mount and shared file

First df shows the root disk, not EFS — the file lands locally. After mount -t efs, the second df confirms EFS, and the file is recreated correctly. A common mistake, worth narrating live.



Instance 2 terminal reading the file from Instance 1

The same mount point on the second instance lists and reads the file created on Instance 1, proving both instances share one file system rather than two separate disks.

When a file written on one instance is missing on the other, the mount listing on both instances usually reveals the cause.

- A disk-usage check confirms whether a shared file system is actually mounted there.
- A missing shared mount means the instance is still using its own local disk.
- A name-resolution check confirms the file system address is reachable before mounting.
- The finished lab leaves one S3 object, one file on the EBS volume of Instance 1, and one EFS file visible from both instances.

Removing lab resources afterward follows a safe order so nothing shared or still in use is deleted by mistake.

- Record the object name, volume ID and file system ID before removing anything.

- *Unmount the EBS volume on Instance 1 and the EFS file system on both instances before detaching or deleting them.*

- Delete only the extra EBS volume and file system created for this lab.
- Terminate only the two instances created for this lab, since termination cannot be undone.
- Empty and delete only the bucket created for this lab.
- Leave any shared networking from an earlier session untouched.

Key Takeaways

1. Block Storage: Volumes attached to instances; IOPS & throughput; use cases
2. File Storage: Shared filesystems: NFS & SMB; when file storage beats block
3. Object Storage: Buckets, objects, keys & metadata; the flat namespace
4. Object Storage Mechanics: REST API access; presigned URLs; versioning
5. Choosing a Storage Type: Comparison across performance, sharing, scale & cost

Upcoming Session : Storage Tiering, Backup & DR

Thank You | All the Best